Венчак Ілона, ТК-31

Лабораторна робота з баз даних

Тема: «NoSQL у системі управління навчальними курсами»

1) Види NoSQL баз даних

Тема роботи: використання NoSQL у системі навчальних курсів. У цій роботі я порівняла PostgreSQL і MongoDB на однакових даних та виконала запити в Neo4j.

У реляційних базах дані зберігаються в таблицях. NoSQL-бази можуть зберігати їх інакше: як пари ключів і значень, документи, групи стовпців або графи. Базу потрібно вибирати під конкретні дані та запити. NoSQL не завжди працює швидше за SQL.

Ключ-значення

Кожне значення має свій ключ. За цим ключем його можна швидко знайти. Наприклад, у Redis можна зберігати сесію користувача або часто потрібні результати. Шукати за самим вмістом значення складніше, ніж за ключем.

Документні бази

Дані зберігаються в документах. Усередині документа можуть бути інші об’єкти та списки. Наприклад, документ курсу в MongoDB може містити всі його модулі й уроки. Це зручно, коли їх потрібно читати разом. Але частина даних може повторюватися, і за їхньою правильністю потрібно стежити.

Широкостовпцеві бази

Приклад такої бази: Cassandra. Дані групуються за ключем, а всередині групи мають визначений порядок. Так можна зберігати багато подій, наприклад дії студента за день. Структуру потрібно планувати під ті запити, які будуть виконуватися. Це не те саме, що зберігати таблицю по стовпцях для аналітики.

Графові бази

У графі є вузли та зв’язки між ними. У Neo4j можна показати, які курси потрібно пройти перед іншими. Також можна знаходити шляхи між курсами. Ця модель зручна для роботи зі зв’язками, але не обов’язково швидша для простого пошуку одного запису.

2) Чому я обрала MongoDB для своєї теми

Моя предметна область: система управління навчальними курсами. У попередній роботі була база PostgreSQL із 19 таблиць. У ній зберігаються користувачі, курси, уроки, матеріали, завдання, групи та результати навчання.

Для цієї роботи я додала нову вимогу. Курс може змінюватися, але група має навчатися за тією програмою, з якої почала. Тому потрібно зберігати старі версії курсу. Кожна версія містить копію програми на момент публікації.

Наприклад, група почала курс із 36 уроків. Пізніше до нової версії додали ще один урок. Для старої групи прогрес потрібно рахувати за її 36 уроками. Для нової групи вже можна використовувати нову програму.

У попередній роботі не дозволялося змінювати опублікований курс через процедури. Тому версії потрібні саме для нової вимоги, а не для виправлення старої помилки.

Для MongoDB я обрала структуру версії курсу: модулі, уроки, матеріали та завдання. Вони зберігаються разом в одному документі. Так усю програму можна отримати одним запитом.

Користувачів, зарахування та оцінки я залишила в PostgreSQL. Там зручно перевіряти зв’язки між записами та зберігати пов’язані зміни разом.

Для тестування MongoDB отримує дані з PostgreSQL. Обидві бази містять ті самі версії. У реальному застосунку ще потрібно було б вирішити, як повторити запис після помилки, щоб дані в базах не відрізнялися.

У цій роботі урок має не більше одного активного завдання. Якщо завдань кілька, програма повідомляє про помилку. Для такого випадку потрібно було б зберігати список завдань.

3) Реалізація в MongoDB

Один документ зберігає одну версію курсу. У ньому є номер версії, дата публікації, кількість уроків і список модулів. Модулі містять уроки, а уроки містять матеріали та завдання.

```
def build_document(cv_id, course_id, version, published_at, published_by, modules):
    return {
        "_id": cv_id, "courseId": course_id, "version": version,
        "publishedAt": published_at, "publishedBy": published_by,
        "lessonCount": lesson_count(modules), "totalMaxScore": total_max_score(modules),
        "modules": modules,
    }
```

Під час завантаження перевіряється структура документа. Також створюється індекс за курсом і номером версії. Він не дозволяє додати дві однакові версії одного курсу. Дані додаються групами, а не по одному документу.

```
    if MONGO_COLLECTION not in db.list_collection_names():
        db.create_collection(MONGO_COLLECTION, validator=VALIDATOR, validationLevel="strict")
    col = db[MONGO_COLLECTION]
    col.create_index([("courseId", ASCENDING), ("version", ASCENDING)], unique=True,
                     name="uq_course_version")
    if col.estimated_document_count() > 0:
        raise SystemExit("Колекція не порожня. Запусти з --drop.")

    pg = psycopg.connect(PG_DSN, autocommit=True)
    meta = pg.execute("""SELECT id, course_id, version, published_at, published_by
                         FROM lms.course_versions ORDER BY id""").fetchall()
    print(f"Версій у PostgreSQL: {len(meta)}", flush=True)
    t0, batch, done = time.perf_counter(), [], 0
    for cv_id, course_id, version, published_at, published_by in meta:
        modules = assemble(pg.execute(FLAT_VERSION_SQL, (cv_id,)).fetchall())
        batch.append(build_document(cv_id, course_id, version, published_at, published_by,
modules))
        if len(batch) >= a.batch:
            col.insert_many(batch, ordered=False)
            done += len(batch)
            batch = []
            print(f"  {done}/{len(meta)} ({time.perf_counter() - t0:.0f} c)", flush=True)
```

Перевірка структури не забороняє змінювати документ. У цій реалізації старі версії не змінюються через код програми. Для нової програми курсу створюється нова версія.

4) Реалізація тих самих даних у SQL

У PostgreSQL одна версія курсу зберігається у п’яти таблицях: версії курсів, модулів, уроків, матеріалів і завдань. Між ними є зовнішні ключі.

```
CREATE TABLE lms.course_versions (
    id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    course_id bigint NOT NULL REFERENCES lms.courses(id),
    version integer NOT NULL CHECK (version > 0),
    published_at timestamptz NOT NULL DEFAULT clock_timestamp(),
    published_by bigint NOT NULL REFERENCES lms.users(id),
    -- Порахувано під час публікації (як lessonCount / totalMaxScore у документі).
    lesson_count integer NOT NULL CHECK (lesson_count >= 0),
    total_max_score numeric(9,2) NOT NULL CHECK (total_max_score >= 0),
    UNIQUE (course_id, version)
);

-- source_*_id навмисно БЕЗ FK: це знімок, а не жива залежність.
CREATE TABLE lms.module_versions (
    id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    course_version_id bigint NOT NULL REFERENCES lms.course_versions(id),
    source_module_id bigint NOT NULL,
    title varchar(200) NOT NULL,
    position integer NOT NULL CHECK (position > 0),
    UNIQUE (course_version_id, position)
);
```

Процедура публікації копіює програму курсу за одну транзакцію. Тобто пов’язані записи зберігаються разом. Тригери забороняють змінювати або видаляти опубліковану версію.

```
# Плоский запит структури опублікованої версії (5 таблиць).
FLAT_VERSION_SQL = """
SELECT mv.source_module_id, mv.title, mv.position,
       lv.source_lesson_id, lv.title, lv.content, lv.duration_minutes, lv.position,
       mt.title, mt.url, mt.kind,
       av.source_assignment_id, av.title, av.max_score
FROM lms.module_versions mv
JOIN lms.lesson_versions lv ON lv.module_version_id = mv.id
LEFT JOIN lms.material_versions mt ON mt.lesson_version_id = lv.id
LEFT JOIN lms.assignment_versions av ON av.lesson_version_id = lv.id
WHERE mv.course_version_id = %s
ORDER BY mv.position, lv.position, mt.id
"""
```

Отримані SQL-рядки збираються в Python у таку саму вкладену структуру, як у MongoDB. Окремо перевіряється варіант, у якому PostgreSQL сам формує JSON.

Цикл версіонування через процедури (09c)

Спочатку в моїй схемі була логічна діра. Процедури sp_add_* змінюють лише курс у статусі draft, а повернути опублікований курс у чернетку не було чим. Тому другу версію через процедури випустити було неможливо. Я закрила це файлом 09c_versioning_workflow.sql і не змінювала файли 00–09b.

Тепер цикл такий. Викладач наповнює чернетку й викликає sp_release_course_version. Вона спершу викликає наявну sp_publish_course, яка перевіряє, що є урок і викладач, і переводить курс у published. Потім вона викликає sp_publish_course_version, яка робить знімок. Обидва кроки йдуть в одній транзакції. Щоб змінити програму, викладач викликає sp_reopen_course, і курс повертається в draft. Це можна зробити, лише коли в курсу вже є хоча б одна версія. Після змін знову викликається sp_release_course_version, і з’являється наступна версія. Саму sp_publish_course_version я не змінювала, бо саме її вимірював бенчмарк.

Когорта тепер отримує версію сама: тригер bind_version під час створення когорти підставляє останню версію курсу. Якщо версій немає, тригер видає помилку, а не записує NULL, бо інакше когорта вчилася б за поточною програмою. Змінити версію когорти після створення не можна. Тригер validate_version не дає позначити урок або здати завдання, яких немає у версії когорти. Нова функція progress_percent_by_version рахує прогрес за уроками версії, а sp_finish_enrollment тепер перевіряє уроки й завдання за версією когорти. Для старих когорт без версії лишилася попередня логіка. Функцію progress_percent і view v_enrollments я не змінювала, бо з ними бенчмарк порівнює прогрес за версією.

Файл 09d_demo.sql показує весь цикл. Когорта A, створена на першій версії, має 2 уроки, а когорта B, створена на другій, має 3. Студентка когорти A не може позначити урок, доданий у другій версії. Вона проходить обидва уроки своєї версії, здає завдання й отримує сертифікат, хоча курс уже має другу версію. За поточною програмою її прогрес був би 66,67%, а за версією когорти він 100%. Демо закінчується ROLLBACK, тому його можна запускати повторно.

```
$ docker exec -i course-nosql-lab-postgres-1 psql -U postgres -d lms_check -f 09d_demo.sql
BEGIN
NOTICE:  v1: course_versions.id = 1
NOTICE:  Очікувана помилка: Only an active draft can be structured
NOTICE:  v2: course_versions.id = 2
NOTICE:  Очікувана помилка: Cohort 1 is already bound to course version 1
NOTICE:  Урок 1 з v1 зараховано когорті A
NOTICE:  Очікувана помилка: Lesson 3 is not part of course version 1 of this cohort
NOTICE:  Прогрес: за живою структурою 66.67 %, за версією когорти 100.00 %
NOTICE:  Зарахування 1 завершено, сертифікат LMS-1
DO
```

```
  cohort   | version | lessons_in_version | lessons_live_now
-----------+---------+--------------------+------------------
 Когорта A |       1 |                  2 |                3
 Когорта B |       2 |                  3 |                3
(2 rows)

 id | operation | old_status | new_status |      changed_by
----+-----------+------------+------------+-----------------------
  1 | INSERT    |            | draft      | Адміністратор системи
  8 | UPDATE    | draft      | published  | Олена Коваль
 16 | UPDATE    | published  | draft      | Олена Коваль
 18 | UPDATE    | draft      | published  | Олена Коваль
(4 rows)

 enrollment |  cohort   | version |  status   | progress_live | progress_by_version | certificate_number
------------+-----------+---------+-----------+---------------+---------------------+--------------------
          1 | Когорта A |       1 | completed |         66.67 |              100.00 | LMS-1
(1 row)

ROLLBACK
```

5) Перевірка швидкодії

Тести виконувалися на Mac з процесором M3. PostgreSQL і MongoDB працювали в Docker. Для кожної бази було виділено по 2 CPU та 3 GiB пам’яті. Для кешу кожної бази було встановлено 1 GiB.

Для повного тесту створено 2 000 курсів і 20 000 версій. У SQL це 120 000 версій модулів, 810 000 версій уроків, 1 620 000 матеріалів і 240 000 завдань. Також є 4 000 груп, 100 000 зарахувань та 2 456 577 записів про прогрес.

Зміни між версіями в тестових даних вносив генератор, а не процедури з 09c. Він змінював таблиці уроків, матеріалів і завдань прямими UPDATE та INSERT з вимкненими тригерами (session_replication_role = replica), а знімок кожної версії робив через sp_publish_course_version. Я обрала такий шлях для швидкості: 2 000 курсів по 9 редакцій через reopen, правки й release тривали б набагато довше, а вміст таблиць версій був би той самий. Тому 31,3% розбіжності прогресу — наслідок саме цих змін: у кожній редакції генератор додає урок, перейменовує урок і змінює max_score, а когорти прив’язані до двох останніх версій. Це число показує, що проблема реальна, але не показує, як часто вона траплялася б на справжній платформі.

Спочатку я виконала короткий тест smoke. Потім очистила тестові дані та виконала full. У кожному основному тесті читання було 30 запусків для прогріву і 300 для вимірювання часу.

Основний прогін вибирає дані з 1 000 ключів. Другий прогін, large_pool, може вибирати з усіх 20 000 версій. Це ті самі дані, а не менший набір. Кеш між прогонами не очищався.

Для порівняння використано медіану. Вона менше залежить від окремих дуже повільних запитів, ніж звичайне середнє. p95 показує межу, швидше за яку виконалися приблизно 95% запитів. Я вважала варіант швидшим, якщо медіана краща хоча б у 1,2 раза і p95 не гірший. Інакше суттєвої переваги за цим правилом немає.

```
[18:04:51]   OK: 60 версій ідентичні; документів Mongo=20000, версій PG=20000
[18:05:51]   OK: 60 версій ідентичні; документів Mongo=20000, версій PG=20000
== 7/7 Neo4j (п. 6)
[18:06:40] Імпортовано курсів: 2000, зв'язків REQUIRES: 5940
[18:06:41] CTE медіана 0.59 мс, Neo4j 2.85 мс
```

Перед вимірюваннями перевірено однаковість 60 випадково вибраних версій. В обох повних прогонах перевірка пройшла. Кількість версій у PostgreSQL і документів у MongoDB дорівнює 20 000.

Результати основного прогону

Нижче наведено час читання структури курсу, обчислення прогресу, запису нової версії та пошуку змін між версіями. Час указано в мілісекундах.

Таблиця 1. Сценарій 1. Завантаження структури опублікованої версії курсу (основний прогін)

Варіант	n	медіана, мс	p95, мс	p99, мс	середнє, мс
sql_flat_rows (без збирання)	300	0.67	1.33	1.83	0.75
sql_flat_rows + збирання у Python	300	0.68	1.62	2.62	0.83
sql_jsonb (збирання на сервері)	300	1.43	2.44	3.51	1.54
mongo find_one(_id)	300	1.05	2.15	3.22	1.20

Таблиця 2. Сценарій 2. Прогрес усієї когорти (основний прогін)

Варіант	n	медіана, мс	p95, мс	p99, мс	середнє, мс
sql_існуюча v_enrollments (жива структура)	300	1.36	1.93	3.01	1.46
sql_версії: знаменник через JOIN	300	0.58	0.96	1.99	0.69
sql_версії: готовий lesson_count	300	0.52	1.03	2.14	0.65
гібрид: PG (прогрес) + Mongo (lessonCount)	300	1.36	2.84	3.98	1.54

Таблиця 3. Сценарій 3. Публікація нової версії курсу (запис) (основний прогін)

Варіант	n	медіана, мс	p95, мс	p99, мс	середнє, мс
sql CALL sp_publish_course_version (без тригерів аудиту)	100	12.93	14.41	15.23	13.09
sql CALL sp_publish_course_version (з тригерами аудиту)	100	16.57	24.63	31.67	17.90
mongo insert_one (лише запис, j=true)	100	1.25	5.84	8.51	1.86
mongo наскрізно (читання PG + збирання + insert)	100	14.23	17.75	20.70	14.73
з них: читання живої структури + збирання	100	12.31	14.81	17.07	12.70

Таблиця 4. Сценарій 4. Дифф двох послідовних версій (основний прогін)

Варіант	n	медіана, мс	p95, мс	p99, мс	середнє, мс
sql FULL JOIN двох версій	300	0.37	0.53	0.94	0.40
mongo: 2 документи + дифф у Python	300	1.04	1.35	2.62	1.11

Одночасне читання та розмір баз

Для одночасного читання працювали 8 потоків. Кожен варіант перевірявся протягом 15 секунд. Також виміряно місце, яке займають дані та індекси.

Таблиця 5. Сценарій 6. Конкурентне читання структури версії (основний прогін)

Варіант	n	медіана, мс	p95, мс	p99, мс	середнє, мс	ops/c (паралельно)
sql flat + збирання	29492	3.28	8.73	11.79	4.06	1966
sql jsonb	17338	3.77	38.29	50.61	6.87	1156
mongo find_one(_id)	36584	2.87	6.53	9.77	3.28	2439

Таблиця 6. Сценарій 5. Розмір на диску (основний прогін)

Сховище	Дані	Індекси	Разом
PostgreSQL (5 таблиць)	556.8 МБ	127.0 МБ	683.7 МБ
MongoDB (стиснуто)	95.1 МБ	0.5 МБ	95.6 МБ

Документів: 20000, середній розмір документа: 28.1 КБ, нестиснутий розмір колекції: 549.3 МБ.

Що показали перші три тести

1. Читання структури. SQL зі збиранням у Python має медіану 0,68 мс, а MongoDB 1,05 мс. Тут SQL швидший. Але якщо SQL сам формує JSON, час зростає до 1,43 мс. Припущення, що спосіб збирання впливає на результат, підтвердилося.

2. Прогрес групи. SQL за версією курсу займає 0,58 мс. Якщо кількість уроків уже збережена, час становить 0,52 мс. Запит до поточної програми курсу та варіант зі зверненням до двох баз займають по 1,36 мс. Припущення підтвердилося: для цього запиту достатньо SQL, а друга база не прискорює його.

3. Запис версії. Сам запис готового документа в MongoDB займає 1,25 мс. SQL без аудиту займає 12,93 мс, а з аудитом 16,57 мс. Але якщо перед записом у MongoDB ще прочитати SQL-дані та зібрати документ, виходить 14,23 мс. За обраним правилом це без суттєвої переваги над SQL без аудиту.

Результати другого прогону

У другому прогоні ключі вибиралися з більшого набору. Для запису було 50 повторів замість 100 у першому прогоні. Основні тести читання знову мали по 300 вимірювань.

Таблиця 7. Сценарій 1. Завантаження структури опублікованої версії курсу (large_pool)

Варіант	n	медіана, мс	p95, мс	p99, мс	середнє, мс
sql_flat_rows (без збирання)	300	0.68	1.33	2.61	0.79
sql_flat_rows + збирання у Python	300	0.67	1.45	4.03	0.81
sql_jsonb (збирання на сервері)	300	1.44	2.44	3.01	1.57
mongo find_one(_id)	300	0.92	1.88	3.73	1.09

Таблиця 8. Сценарій 2. Прогрес усієї когорти (large_pool)

Варіант	n	медіана, мс	p95, мс	p99, мс	середнє, мс
sql_існуюча v_enrollments (жива структура)	300	1.34	1.62	2.08	1.38
sql_версії: знаменник через JOIN	300	0.57	0.79	1.00	0.59
sql_версії: готовий lesson_count	300	0.53	0.76	0.86	0.54
гібрид: PG (прогрес) + Mongo (lessonCount)	300	1.02	1.76	2.52	1.14

Таблиця 9. Сценарій 3. Публікація нової версії курсу (запис) (large_pool)

Варіант	n	медіана, мс	p95, мс	p99, мс	середнє, мс
sql CALL sp_publish_course_version (без тригерів аудиту)	50	13.35	16.07	17.15	13.89
sql CALL sp_publish_course_version (з тригерами аудиту)	50	16.94	19.38	20.70	17.48
mongo insert_one (лише запис, j=true)	50	1.42	2.27	2.85	1.53
mongo наскрізно (читання PG + збирання + insert)	50	14.30	16.58	17.38	14.51
з них: читання живої структури + збирання	50	12.20	14.19	15.47	12.44

Таблиця 10. Сценарій 4. Дифф двох послідовних версій (large_pool)

Варіант	n	медіана, мс	p95, мс	p99, мс	середнє, мс
sql FULL JOIN двох версій	300	0.41	0.98	2.00	0.49
mongo: 2 документи + дифф у Python	300	1.46	3.02	6.10	1.77

Другий прогін і висновки для решти тестів

Таблиця 11. Сценарій 6. Конкурентне читання структури версії (large_pool)

Варіант	n	медіана, мс	p95, мс	p99, мс	середнє, мс	ops/c (паралельно)
sql flat + збирання	17936	3.59	9.27	13.08	4.45	1794
sql jsonb	11157	4.09	37.92	47.78	7.15	1116
mongo find_one(_id)	23751	2.94	6.75	9.96	3.37	2375

Таблиця 12. Сценарій 5. Розмір на диску (large_pool)

Сховище	Дані	Індекси	Разом
PostgreSQL (5 таблиць)	560.0 МБ	127.7 МБ	687.8 МБ
MongoDB (стиснуто)	95.2 МБ	0.5 МБ	95.7 МБ

Документів: 20000, середній розмір документа: 28.1 КБ, нестиснутий розмір колекції: 549.3 МБ.

Для перших трьох тестів загальний висновок не змінився. SQL зі збиранням має 0,67 мс проти 0,92 мс у MongoDB. Повний запис через MongoDB займає 14,30 мс, а SQL без аудиту 13,35 мс. Тут знову немає суттєвої переваги за обраним правилом.

4. Зміни між версіями. SQL швидший в обох прогонах: 0,37 проти 1,04 мс та 0,41 проти 1,46 мс. Припущення підтвердилося. SQL повертає тільки зміни, а для MongoDB потрібно отримати два документи й порівняти їх у Python. У цьому тесті порівнюються поля уроків та їхній модуль, але не матеріали й завдання.

5. Місце на диску. MongoDB займає приблизно 95,6-95,7 MiB, а SQL-таблиці з індексами 683,7-687,8 MiB. Припущення про слабке стиснення не підтвердилося. У тестовому тексті один md5-рядок повторюється 12 разів, тому він добре стискається. На розмір також впливають індекси та спосіб зберігання. Журнали запису не враховувалися.

Різниця в розмірі майже повністю пояснюється штучним текстом уроків. Генератор записує в текст уроку той самий md5, повторений 12 разів, тобто 32 унікальні символи на 384. MongoDB стискає блоки даних і на таких повторах дуже виграє. PostgreSQL стискає значення лише в довгих рядках (TOAST, приблизно від 2 КБ), тому ці 384 символи зберігаються як є. Це видно з моїх же вимірювань: без стиснення колекція MongoDB

займає 549,3 МБ, тобто приблизно стільки ж, скільки дані PostgreSQL без індексів (556,8 МБ). На реальних текстах уроків стиснення було б набагато слабшим, і різниця стала б значно меншою. Тому я не роблю висновку, що MongoDB займає в сім разів менше місця.

6. Одночасне читання. У першому прогоні MongoDB і SQL зі збиранням не мають суттєвої різниці за обраним правилом. У другому MongoDB швидша: 2,94 проти 3,59 мс, і p95 також кращий. MongoDB швидша за SQL JSON в обох прогонах. Припущення про вплив Python окремо не доведено, бо навантаження на процесор клієнта не вимірювалося.

Чому результати відрізняються

Я переглянула плани запитів, щоб зрозуміти, як бази знаходять дані. Для PostgreSQL використано EXPLAIN (ANALYZE, BUFFERS), а для MongoDB executionStats.

Під час пошуку документа за _id MongoDB переглядає один ключ індексу й один документ. PostgreSQL теж використовує індекси. У перевіреному прикладі SQL повертає 76 рядків, які потім збираються в Python.

Коли PostgreSQL сам створює JSON, він додатково збирає вкладені списки на сервері. Це узгоджується з більшим часом цього варіанта. Але один план запиту не пояснює всі можливі причини різниці.

Для прогресу за поточною програмою курсу було 1 545 звернень до сторінок у пам’яті. Для версії курсу через JOIN було 239, а з готовою кількістю уроків 219. Тобто версійні запити виконують менше роботи.

Під час порівняння версій SQL повернув два змінені рядки й відкинув 37 незмінених. Це пояснює, чому передавання лише змін може бути швидшим за читання двох повних документів.

Для запису й одночасного навантаження окремі плани не збиралися. Тому за цими тестами можна порівняти час, але не можна точно сказати, скільки витрачено на процесор, очікування або журналювання.

Під час запису SQL чекає на завершення транзакції. MongoDB використовує w=1 і j=true. Тест запису MongoDB працює в окремій колекції без перевірки структури документа. Тому умови перевірки даних не повністю однакові.

Через це 1,25 мс у першому прогоні (1,42 мс у другому) — це запис без перевірки схеми. З валідатором запис міг би бути трохи повільнішим, але я цього не вимірювала: окремого прогону з валідатором немає. На висновок про паритет наскрізного запису це майже не впливає: з 14,23 мс 12,31 мс займають читання поточної програми з PostgreSQL і збирання документа, а не сам insert.

Для 5 000 зарахувань я також порівняла прогрес за поточною програмою та за версією групи. Значення відрізнялися у 1 566 випадках, або 31,3%. Це показує, чому важливо зберігати програму, за якою почала навчатися група.

Після тестів кількість версій залишилася 20 000. Усі п’ять тригерів аудиту ввімкнені, а тимчасові тестові дані прибрані.

6) Робота з графовою базою Neo4j

Другою NoSQL-базою я обрала Neo4j. У ній курс є вузлом, а REQUIRES показує, який інший курс потрібно пройти перед ним. Завантажено 2 000 курсів і 5 940 зв’язків.

Я виконала створення курсу та зв’язку, пошук курсу, зміну назви й видалення. Для цих дій використано окремий тестовий курс.

1. CREATE: новий курс і зв'язок (MERGE - повтор не дублює)

```
MERGE (d:Course {id: -1}) SET d.code = 'DEMO-1', d.title = 'Demo course' WITH d MATCH
(p:Course {code: $prereq}) MERGE (d)-[:REQUIRES]->(p) RETURN d.code AS code, p.code AS
requires
```

Параметри: {'prereq': 'BENCH-00001'}

Рядків: 1, час: 181.1 мс. Перші рядки:

```
{'code': 'DEMO-1', 'requires': 'BENCH-00001'}
```

2. READ: курс за кодом

```
MATCH (c:Course {code: $code}) RETURN c.id AS id, c.code AS code, c.title AS title
```

Параметри: {'code': 'BENCH-02000'}

Рядків: 1, час: 65.3 мс. Перші рядки:

```
{'id': 2000, 'code': 'BENCH-02000', 'title': 'Bench course 2000'}
```

4. UPDATE: зміна властивості

```
MATCH (d:Course {id: -1}) SET d.title = 'Demo course (updated)' RETURN d.title AS title
```

Рядків: 1, час: 54.6 мс. Перші рядки:

```
{'title': 'Demo course (updated)'}
```

5. DELETE: видалення демо-вершини разом зі зв'язками

```
MATCH (d:Course {id: -1}) DETACH DELETE d RETURN count(*) AS deleted
```

Рядків: 1, час: 86.1 мс. Перші рядки:

```
{'deleted': 1}
```

Після перевірки тестовий курс видалено разом із його зв’язком. Усі запити завершилися без помилок.

Пошук зв’язків між курсами

6. Транзитивні пререквізити (до 6 кроків)

```
MATCH (c:Course {code: $code})-[:REQUIRES*1..6]->(p:Course) RETURN DISTINCT p.code AS
prerequisite ORDER BY prerequisite
```

Параметри: {'code': 'BENCH-02000'}

Рядків: 98, час: 111.0 мс. Перші рядки:

```
{'prerequisite': 'BENCH-01835'}
{'prerequisite': 'BENCH-01839'}
{'prerequisite': 'BENCH-01842'}
{'prerequisite': 'BENCH-01843'}
{'prerequisite': 'BENCH-01846'}
{'prerequisite': 'BENCH-01849'}
{'prerequisite': 'BENCH-01850'}
{'prerequisite': 'BENCH-01853'}
```

7. Зворотний обхід: які курси залежать від цього (до 3 кроків)

```
MATCH (p:Course {code: $code})<-[:REQUIRES*1..3]-(c:Course) RETURN count(DISTINCT c) AS
dependents
```

Параметри: {'code': 'BENCH-00001'}

Рядків: 1, час: 59.6 мс. Перші рядки:

```
{'dependents': 31}
```

8b. Найкоротший шлях між курсами

```
MATCH (a:Course {code: $a}), (b:Course {code: $b}) MATCH path = shortestPath((a)-
[:REQUIRES*..30]->(b)) RETURN [n IN nodes(path) | n.code] AS route, length(path) AS hops
```

Параметри: {'a': 'BENCH-02000', 'b': 'BENCH-01835'}

Рядків: 1, час: 87.3 мс. Перші рядки:

```
{'route': ['BENCH-02000', 'BENCH-01971', 'BENCH-01945', 'BENCH-01916', 'BENCH-01890', 'BENCH-
01861', 'BENCH-01835'], 'hops': 6}
```

Для курсу BENCH-02000 знайдено 98 різних курсів, які потрібні перед ним, якщо шукати не далі шести кроків. Від BENCH-00001 залежать 31 різний курс у межах трьох кроків.

Також знайдено найкоротший шлях від BENCH-02000 до BENCH-01835. Він має шість зв’язків. Додатково перевірено пошук курсів без передумов і курсів, доступних після проходження перших 30.

Висновок

Я розглянула чотири типи NoSQL-баз і додала до системи курсів зберігання версій програми. Однакові дані реалізовано в MongoDB та у п’яти таблицях PostgreSQL. Для порівняння використано 20 000 версій і шість різних тестів.

Результати показали, що немає однієї бази, яка швидша в усіх випадках. SQL зі збиранням у Python швидше читає окрему версію. SQL також швидше знаходить зміни між версіями. MongoDB швидше записує готовий документ і займає менше місця на диску.

Якщо врахувати читання SQL-даних, збирання документа й запис у MongoDB разом, значної переваги над SQL без аудиту немає. Під час одночасного читання результат залежить від того, як SQL формує відповідь і з якого набору вибираються версії.

MongoDB зручно використовувати для збереженої програми курсу, яку потрібно отримувати цілком. Користувачів, зарахування та оцінки доцільно залишити в PostgreSQL.

У Neo4j я перевірила основні дії з даними та пошук залежностей між курсами. Для 100 курсів додатково порівняно пошук передумов із SQL. Медіана SQL становила 0,59 мс, Neo4j 2,85 мс. На цьому наборі SQL швидший, хоча графові запити зручно описують зв’язки.

Перевірено однаковість 60 вибраних версій, а не всіх 20 000. Для графа звірялися кількості знайдених передумов, а не повні списки. Холодний кеш, робота після збоїв і копіювання даних між кількома серверами не тестувалися. Тому висновки стосуються саме цих даних і запитів.

Обмеження

Під час редагування курсу (draft) завдання можна видалити через soft delete, а в знімку старої версії воно лишається. Тоді студенти старої когорти не зможуть його здати, і sp_finish_enrollment не дасть їм завершити курс.

sp_grade порівнює оцінку з поточним max_score завдання, а не з тим, що збережений у версії когорти.

Поки курс у draft, нові когорти не створюються, а в наявні не можна зарахуватися, бо sp_create_cohort і sp_enroll вимагають статусу published.

Холодний кеш і реплікацію я не тестувала. Обидві бази працювали на одному вузлі, а робочий набір вміщувався в кеш. Прогін large_pool лише розширює набір ключів і не є тестом холодного кешу.

Запис у MongoDB я вимірювала в колекції без валідатора, тому гарантії запису в двох базах не повністю однакові.